FORGEKERNEL: SELF-EVOLVING KERNELS CATCHING AND SURPASSING EXPERTS AND AGENTS

Anonymous authors
Paper under double-blind review

ABSTRACT

Frontier coding agents now write GPU kernels that rival expert ones: left to run, even a generic agentic loop beats hand-tuned references on many operators. Yet every such loop stops at a ceiling of its own making. It settles a kernel's architecture in its first few attempts and spends the rest of the run tuning beneath it, and each run starts from the same knowledge as the last and discards what it measured.

We present ForgeKernel, a self-evolving kernel agent built to break that ceiling. It reads a reference kernel's architecture from hardware behavior alone — running, timing and profiling a sealed binary, never its source — and turns it into an architecture-first curriculum that a coding agent implements and that is reopened whenever measurement shows the architecture, not the tuning, is the bottleneck. Every accepted kernel is sealed and promoted to the next campaign's reference, so optimization continues past the best known implementation, and every finding certified by a hardware probe enters a knowledge base shared across operators.

Across 27 operators on Hopper and Blackwell, ForgeKernel matches or beats 18 of 22 expert-library kernels, including FlashAttention-4, cuDNN and DeepGEMM, and all 7 published kernels of three kernel-generation systems, the latter by up to +170.6% (+39.4% in geometric mean). On DeepSeek-V4.1's sparse-MLA prefill and fused attention kernels, released 15 days before this submission with no paper or second implementation, it comes within 1.3% and 3.7%. With only its profiler swapped, it leads the best public submission on all 8 level-4 CANN-Bench operators on Ascend, by +33.1% in geometric mean. Promoting the best known kernel to the reference, a generic loop's included, gains up to a further +9.9%.

1 INTRODUCTION

Modern machine-learning systems demand ever more of GPU kernel performance, and new operators (sparse attention indexers (DeepSeek-AI, 2025b), gated delta networks (Yang et al., 2024), latent-attention variants (DeepSeek-AI, 2024)) keep arriving. The good news is that large language models can already write efficient GPU kernels themselves. A body of prior work has built agents dedicated to kernel optimization (Chen et al., 2025; Wang et al., 2025a; Dong et al., 2025; Wei et al., 2025; Han et al., 2026; Luo et al., 2026), and in some settings those agents beat expert implementations. More striking is an open-source result (vincent317, 2026): a frontier model (Claude Fable 5.1) using nothing more than the `/goal` mode of Claude Code already matches expert kernels on many tasks.

As the minimal form of an operator agent, that result makes us ask how an operator-specific agent should be designed on top of today's frontier models. Such an agent exists to close the model's remaining capability gaps rather than to gild the lily where the model already delivers, and the procedures it carries are provisional: as agentic kernel data accumulates in training corpora the model internalizes them, and the harness is dismantled piece by piece, because what remains turns into over-guidance. Kernel Design Agents (Liu et al., 2026) is the representative case, a harness already thinned from a system dense with optimization-detail guidance into a lightweight workflow that `/goal` traces show the model using and improving on its own. None of this means the operator agent has lost its value. We measured where `/goal` stops (§2) and found two gaps worth closing. On some operators it still falls short of the best human implementation, and the obstacle is architectural rather than tactical: the architecture is settled in the first few attempts, and the local tuning that

Reviewers: please read the Reviewer Guidelines (iclr.cc/Conferences/2027/ReviewerGuidelines) and the AI Policy for Reviewers (iclr.cc/Conferences/2027/AIPolicyForReviewers). If you used AI to expand, edit, or polish your review, please provide the input text to the LLM. Better still, consider skipping the LLM and submitting your original text: we, and the authors, are much more interested in your unedited thoughts than in what an LLM has to say. AI-assisted or not, you are putting your name and reputation behind your review: LLM-generated falsehoods, hallucinations or misrepresentations are subject to disciplinary action, which may include desk-rejecting all papers you have authored.

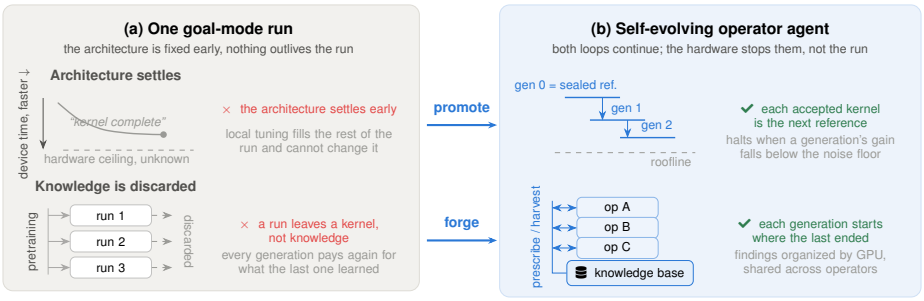


Figure 1: The two mechanisms of self-evolution. (a) A goal-mode run settles its architecture early and stops at a plateau of unknown distance from the ceiling, discarding what it measured. (b) ForgeKernel promotes each accepted kernel to the next reference (top) and forges certified findings into a knowledge base (bottom).

fills the rest of the run cannot change it. On the operators where it does beat human experts, nothing says it has reached the hardware limit.

These observations led us to ForgeKernel, a self-evolving operator agent system (Figure 1), built to keep assisting the base model's progress and to explore the hardware limit until the model can settle every operator on its own. Two core designs carry that goal. The first is analysing a reference implementation and reproducing its performance, which starts every operator at the current state of the art; while analysing it, ForgeKernel forms its own judgment and may accept or improve on the reference's architectural decisions, so the architecture is adjustable as the campaign runs rather than fixed at the outset. The second is self-evolution by feeding the result back: ForgeKernel takes a kernel it wrote itself as the new reference, analyses it again, and surpasses it again. That is also what keeps the search open past the state of the art, with the gains shrinking toward the hardware roofline until a generation's gain falls below the measured noise floor, at which point the loop stops and the plateau it reached is itself an estimate of the ceiling. Neither design needs the reference's code, only the ability to profile it, which lets domains with little open-source code, training among them, share in the dividend.

Self-evolution is affordable only with a third thing: knowledge that outlives a run. A goal-mode run leaves behind a kernel, not knowledge, so inside a self-evolution loop every generation pays again for what the previous one already learned. ForgeKernel is stateful by design. Findings that survive measurement accumulate in a knowledge base, organized by GPU and shared across operators and shapes, so each generation starts where the previous one ended rather than where the model's pretraining did.

Across 27 operators and 29 input shapes on Hopper and Blackwell, ForgeKernel matches or beats its expert reference on 25 of the 29 cells, and matches or beats all 7 published kernels of three recent generation systems. Against the `/goal` agent the comparison splits along the agent's own outcome rather than along the operator: where the agent fell short of its reference, ForgeKernel reached it on every one of those kernels, and where the agent led, its converged kernel became the next reference and was passed, for a further +9.5% over kernels whose author had stopped improving them.

ForgeKernel therefore contributes three things, in the order it applies them: reading a kernel's architecture off a reference it never reads, a knowledge base of probe-certified hardware mechanisms that transfers across operators sharing a GPU, and self-evolution by promoting its own accepted kernel to the next campaign's reference.

2 How Far a Frontier Agent Gets on Its Own

Before adding any machinery we measured what the model alone can do. The */goal agent* is Claude Fable 5.1 driven by the unmodified Claude Code CLI in its default `/goal` mode, up to 24 hours per kernel, with a fixed prompt: the operator signature, the input shapes, and one H100. The only thing withheld is code. It cannot read the reference's source or copy code from the web, but it can run, time, profile and investigate the reference with the same tools ForgeKernel's arch

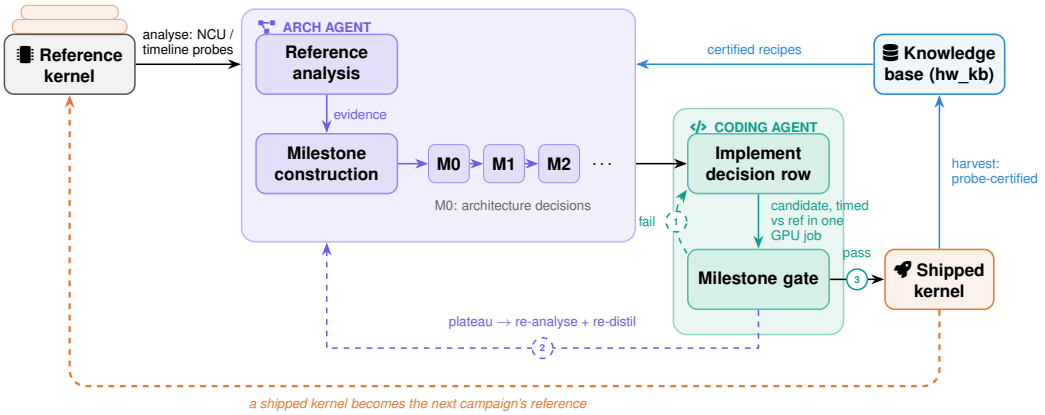


Figure 2: ForgeKernel architecture. The arch agent turns a reference kernel's hardware-observable behavior into a milestone sequence; the coding agent implements one decision row at a time, each timed against the reference in the same GPU job. Gate outcomes fan out three ways: retry the row, re-analyse and revise the curriculum, or ship.

agent later uses, so what separates the two is machinery rather than information. Its objective is open-ended and its loop stateless across sessions, so every number below is a converged reading rather than a first draft, and nothing it learns outlives one run. A kernel counts only if its per-tensor error is within 2× the expert reference's own; we audited each for accuracy-for-speed trades. The ten tasks span attention, GDN and the members of the DSA sparse-attention chain, forward and backward, with references from FlashAttention-4, FlashInfer, FlashQLA, DeepGEMM, FlashMLA and cuDNN-frontend. Table 5 (Appendix I) is the ceiling we start from.

Ten tasks, seven wins. The agent beats its reference on 7 of the ten tasks, for a geometric-mean +17% over the set. Its wins include FlashAttention-4 on GQA attention forward and backward, all three cuDNN-frontend references of DSA's backward chain, cuDNN-frontend's radix top-k, and FlashInfer's fused GDN prefill kernel — a young operator with little public precedent, independently re-verified on a second H100 and a second FlashInfer version. The three it does not beat are DSA's fp8 indexer forward, DSA sparse attention forward and GDN prefill backward, the first two against the most heavily engineered references in the set. The shortfalls follow neither the operator nor the direction of the pass: the DSA sparse-attention chain alone holds a forward loss and a backward win.

The fixed point. Whether the `/goal` agent ends above its reference or below it, it ends the same way: at a converged kernel its own loop can no longer improve. Every row of Table 5, the wins as much as the losses, is where an unbounded optimizer stopped. That fixed point, not the reference, is what the rest of this paper measures against. Two things end there. The search ends, because the agent settles its architecture in the first few attempts and the local tuning that fills the rest of the run cannot reopen it. And the knowledge ends, because a stateless session discards every hardware fact it paid to measure. §3 is built to continue both.

3 Method

3.1 Overview

A kernel's performance is set by two layers: *architectural decisions* that fix its global structure — tiling, pipeline depth, warp roles, tensor layouts — and *tactical knowledge* that executes that structure. On Hopper and Blackwell the first sets a ceiling the second does not raise (NVIDIA Corporation, 2022; 2026b; 2024; Thakkar et al., 2026; Shah et al., 2024) (Appendix A). The architectural layer is also observable without source, through profiler signatures that targeted probes disambiguate (§3.2).

A campaign takes an operator, an input shape, a sealed reference R and a target gain τ relative to it, and returns a kernel K with the milestone curriculum $M_0 \to M_k$ that produced it. Writing t_K for device time, the gain $g(K, R) = t_R/t_K$ is always measured with both kernels timed in the same GPU job, and the campaign succeeds when K passes the operator's correctness judge and $g(K, R) \geq \tau$. Self-evolution (§3.5) iterates exactly this map, with $R_{n+1} = K_n$, and stops when a confirmed gain falls to the measurement noise floor, $g(K_{n+1}, K_n) - 1 < \epsilon$ with $\epsilon = 1\%$–5%. Nothing in the map depends on who wrote R, which is why an expert library, another system's kernel and ForgeKernel's own output are all admissible references.

Following the two layers, an *arch agent* analyses the reference, writes and revises the curriculum and diagnoses the trajectory, and a *coding agent* writes every line of kernel code. Three artifacts mediate them: the per-operator *milestone*, which stores decisions and measured targets but no implementation body; the *hardware knowledge base* of operator-agnostic mechanisms, the only artifact that contains code; and the *reference-analysis policy* that turns the first into the second. The harness and prompts hold no operator-specific information.

Two rules govern the design: *no code crosses the boundary*, which a deterministic driver enforces outside the prompts by shipping the reference as a payload only the GPU job unseals; and *an agent's own claim is not evidence*, so every number, gate and knowledge-base entry rests on a measurement (Appendices B and C).

3.2 Reference Analysis and Milestone Construction

Reference analysis recovers the reference's architecture from hardware-observable behavior. An NCU pass yields launch structure, occupancy, pipeline utilization and stall decomposition, from which the arch agent infers kernel count, warp roles, pipeline depth and the limiting resource; serial latency-bound kernels also receive a timeline probe, and what the instruments cannot resolve stays an explicit open question. The arch agent turns these observations into an ordered milestone sequence $M_0 \to M_k$ of decisions, each stated in natural language with its action, purpose, trigger, cost and dependence. Natural language is the carrier because references share no implementation language — they are written in CUDA C++ (Thakkar et al., 2026), Triton (Tillet et al., 2019), TileLang (Wang et al., 2025b) or ThunderKittens (Spector et al., 2024), or ship only as binaries (Chetlur et al., 2014; NVIDIA Corporation, 2026a) — so a decision read off DeepGEMM's CUDA is realized in the coding agent's CuTe DSL, and a recipe forged for one operator can be prescribed to another. A decision without a trigger, the profiler symptom that activates it, is an unsupported claim.

M_0 carries the decisions that matter most — the algorithm contract (decomposition, dataflow, intermediate-tensor ABI, precision) and the hardware architecture (tile shapes, warp roles, register budget, shared-memory layout, MMA atoms) — checked against a *resource-topology contract* transcribed from the reference signature; M_1 through M_k then tune levers such as stage counts, swizzle modes and wait depths on that topology, since a late topology change forces a liveness-plan retrofit. M_0 comes first but is not final: a ceiling that requires a topology change is an M_0 defect, and the arch agent reopens it (§3.3). Every milestone closes with a gate of correctness lines, performance lines with provenance, and a *mechanism-verification line* — a profiler-anchored predicate that separates a landed mechanism from a coincidental speedup. An unresolved line must be discharged as DEFERRED, INFEASIBLE or N/A with its own evidence (Appendix B), and a decision graph exhausted below target returns to the arch agent as a milestone defect.

3.3 Nested Loops of ForgeKernel

In the inner loop the coding agent implements one decision row at a time, and a single GPU job evaluates it through an independent FP32 correctness check, paired timing and profiling; correctness failures are reverted and M_0 is evaluated as one architectural unit. The outer loop is what a single-episode agent lacks: the arch agent re-enters after 3 iterations without a new best, on decision-graph exhaustion or at a gate impasse, prices levers and runs verdict probes to isolate individual and coupled effects, and revises the curriculum. Additive patches resume in place; foundation patches restart from the earliest dependent decision, which is how a topology decision taken in M_0 is reopened instead of tuned beneath. A candidate is accepted only when its same-job gain clears the noise floor 1%–5%, and a patch that no measurement can credit is reverted and recorded as a no-go.

3.4 KNOWLEDGE FORGING AND REUSE

Each knowledge-base recipe records a mechanism, its trigger symptom, a minimal runnable example and known gotchas, organized by GPU architecture. Agent-readable provenance cites only documentation, public API symbols and probe verdicts; reference-specific names stay in a human-only archive that deterministic lint keeps separate. A recipe therefore retains no trace of the reference it was forged against, which is what permits one written during an attention-backward campaign to be read by a GEMM campaign that shares only the hardware.

Admission requires measurement: a proposal from either agent enters the base only after a GPU probe on the target architecture certifies it. A campaign that meets its target does not establish every explanation offered within it; certification separates the mechanism that mattered from those that merely accompanied it, so what accumulates is hardware behavior rather than a record of one successful configuration. Prescription is the return path: the arch agent cites recipes by name in the rows that require them, and campaign ledgers record both the recipes a campaign invents and the foreign recipes its delivered kernel lands, which makes the base's contribution auditable rather than asserted. Dead ends accumulate the same way, in a no-go table keyed by symptom and retry precondition.

3.5 SELF-REFERENCE FOR ITERATIVE PERFORMANCE IMPROVEMENT

Because a reference is used only as a measurable binary, a kernel ForgeKernel has accepted fills that role exactly as a vendor binary does. ForgeKernel seals every promoted kernel — its own or the `/goal` agent's — and supplies it to the next campaign as its architecture reference. That campaign repeats the analysis against it: it rereads the NCU signature, reanchors targets and mechanism lines to the promoted reference, and applies the per-phase target gain +5%. Source does not cross a generation boundary; the knowledge base does, which is why the two feedback edges of Figure 2 are not redundant. Promotion raises the performance anchor and the knowledge base lowers the cost of reaching it, and the chain stops when a confirmed per-phase gain falls below the same-job noise floor or the budget ends.

4 EXPERIMENTS

4.1 EXPERIMENTAL SETUP

Hopper experiments run on H100 80 GB (SXM, 132 SMs) and Blackwell on B200 (148 SMs) nodes; the coding agent writes CuTe DSL. *Efficiency* is device time relative to the reference, both timed in the same job, and every candidate clears an operator-specific correctness judge before it is timed. The arch agent runs Claude Fable 5.1 and the coding agent Claude Opus 5, both stock Claude Code agents. A campaign stops on reaching its target gain even when more is available. Figure 3 therefore measures whether a sealed binary can be matched, not how far past it the system goes; where the `/goal` agent beat its reference by more, its kernel is the better starting point, and §4.3 promotes it on two such tasks. The Reproducibility Statement and Appendix G give the confirmation protocol, the judges, the pinning and the instruments; Appendix H, the cost of every campaign.

Gains are not interchangeable across the three subsections that follow: over a sealed reference in §4.2, over the previous generation in §4.3 (whose cumulative column compounds them), and over one fixed expert library in §4.4.

4.2 REACHING EXPERT LIBRARIES, BEATING SOTA SYSTEMS

Every campaign in this block is given exactly what the `/goal` agent had: it can run and profile the reference and is denied only its source. The comparisons therefore isolate the machinery of §3, not the information. They also share one harness — the same driver, milestone templates, judges and measurement protocol, on Hopper and on Blackwell, for GEMM and for attention, against a library and against another system's kernel. Nothing in it is specialized to an operator or a GPU; one campaign differs from the next only in its task card (operator, shape, sealed reference, target) and in the state of the knowledge base when it starts.

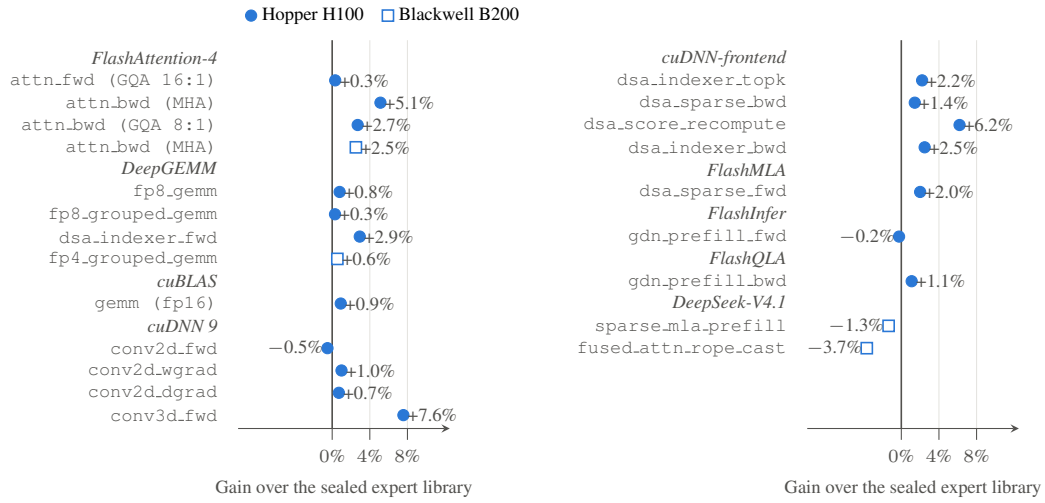


Figure 3: ForgeKernel against expert libraries: one dot per operator, shape and GPU, best campaign per cell, grouped by the sealed reference's library. *Gain* is the post-stop confirmed speedup over that reference; the vertical line is parity. Shapes: Table 3.

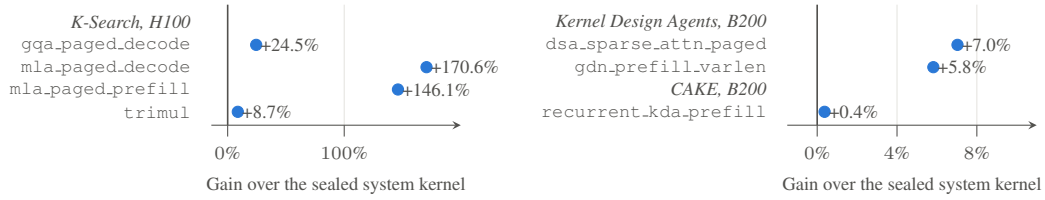


Figure 4: ForgeKernel against the kernels of SOTA generation systems (K-Search (Cao et al., 2026) on Hopper; Kernel Design Agents (Liu et al., 2026) and CAKE (Ye et al., 2026) on Blackwell), best campaign per cell; axes as in Figure 3, but each GPU on its own scale.

Against the strongest human experts. Against expert libraries, ForgeKernel matches or beats the reference on 18 of the 22 cells of Figure 3 and finishes within 1% of it on 2 more, for a geometric-mean +1.6% and a best of +7.6%. This holds on both GPU generations and in every operator family: the GEMM and convolution cells, against years of tuning in cuBLAS, DeepGEMM and cuDNN, land between −0.5% and +7.6%; FlashAttention-4 is beaten on its Hopper forward and backward and on its Blackwell backward; and every backward kernel of the DSA chain beats cuDNN-frontend, by up to +6.2%. The margins are small because of what is being tested — among the most heavily engineered kernels in the field, observed only as binaries. Two cells stop less than 1% below their reference; one of them, FlashInfer's GDN prefill (−0.2%), is a task §4.3 returns to. The two furthest are the newest: DeepSeek-V4.1's sparse MLA prefill (−1.3%) and its kernel fusing RMSNorm, both RoPE passes, sparse attention and an FP8 cast into one launch (−3.7%), which DeepSeek released with V4.1 (DeepSeek-AI, 2025c) 15 days before this submission with no paper describing them and no other library reimplementing them. What ForgeKernel needs from a reference is its behavior on the GPU, which a kernel days old exposes as fully as one studied for years.

Against the strongest kernel agents. Reaching the expert libraries does not show that ForgeKernel is better than other automated systems, which might reach them too. Figure 4 tests that directly, with the strongest published kernels of three generation systems as sealed references: K-Search's (Cao et al., 2026) winning kernels on its own FlashInfer-Bench workloads and the GPUMode TriMul task on Hopper, and those of Kernel Design Agents (Liu et al., 2026) and CAKE (Ye et al., 2026) on Blackwell. ForgeKernel finishes at or above all 7 of them, with a geometric-mean gain of +39.4% and a best of +170.6%. The margins follow how much headroom each system left. K-Search's four kernels left the most, among them the TriMul kernel its authors report as faster than every top-ranked submission on the GPUMode leaderboard (Cao et al., 2026).

Table 1: Self-evolution by reference promotion: the best known kernel for each task when its chain started is sealed as the next reference with a +5% target. The parenthesis is the promoted kernel's gain over the expert library; *Cumul.* compounds it with the new campaign's gain. The bottom row promotes twice in succession, gen-1 then gen-2.

Best known is the `/goal` agent's: promote it				*Best known is ForgeKernel's: promote our own*			
Operator	Ref. (vs. expert)	Gain	Cumul.	Operator	Ref. (vs. expert)	Gain	Cumul.
gdn_prefill_fwd	agent (+82.6%)	+9.5%	+100%	dsa_sparse_fwd	gen-0 (+2.0%)	+7.2%	+9%
attn_fwd (GQA 16:1)	agent (+22.8%)	+5.0%	+29%	gdn_prefill_bwd	gen-0 (+1.1%)	+2.0%	+3%
Promote ForgeKernel's own kernel twice in succession, left to right							
attn_fwd (GQA 16:1)	gen-0 (+0.3%)	+9.9%	+10%	attn_fwd (GQA 16:1)	gen-1 (+10.3%)	+4.2%	+15%

Kernel Design Agents' KDA-v0.5 kernels (Zou et al., 2026) left less — their authors measure them at 41% and 69% over the winning MLSys 2026 FlashInfer contest submissions on DSA sparse attention and GDN prefill — and ForgeKernel passes both. CAKE's persistent KDA kernel is matched at +0.4%, parity rather than a win.

Cross-architecture test. On Ascend 910C we swapped NCU for msprof and ran the loop as is, with the fastest implementation in Huawei's current operator libraries as each operator's reference. On all 8 level-4 operators of CANN-Bench 1.1.2, the final kernel leads the best public submission on the leaderboard, by +33.1% in geometric mean of speedup (Figure 6, Appendix I).

4.3 Self-Evolution

Parity with a reference is where §4.2 stops, not where the system does. ForgeKernel needs a reference only as a binary to profile, so *any* kernel can serve as one, including a machine-written kernel whose source nobody has studied. Self-evolution is this rule applied repeatedly: take the best kernel known so far for a task, whoever wrote it, promote it to the reference, and run the unchanged pipeline against it. A longer `/goal` run is not the alternative, because its readings are already converged and it carries nothing across sessions (§2); sealing its kernel as the next reference and carrying earlier findings forward are precisely the two mechanisms ForgeKernel adds, and Table 1 measures them.

Where the `/goal` agent leads, its kernel is the reference. On GDN prefill and GQA attention forward the agent's kernel was the best known (Table 5). Sealed and handed to ForgeKernel as the reference, each was passed (Table 1): the results stand +100% above FlashInfer's and +29% above FlashAttention-4's. ForgeKernel sees no more than the agent did, and neither reads expert source. What it brings is what the agent's loop lacks: a knowledge base of probe-certified hardware behaviour paid for by other operators' campaigns (§3.4), and a loop that re-enters at the level of architecture and orchestration when progress stalls, instead of tuning beneath the architecture it chose first. The gain is modest, but the agent's own loop had stopped finding it.

Where ForgeKernel leads, it promotes its own kernel. On DSA sparse-attention forward and the GDN backward, ForgeKernel's kernel was the best known, so we promoted it to the reference of its own task and ran the pipeline again. The sparse-attention chain gained +7.2% over its predecessor, and the GDN backward chain +2.0% (Table 1). ForgeKernel self-evolves with no outside reference at all. Promotion also repeats: on GQA attention forward we promoted ForgeKernel's own kernel twice in succession, each generation sealed as the next one's reference, ending +15% above FlashAttention-4 (Table 1).

Why promotion keeps finding gains. Promotion acts like a restart in iterated local search: each generation starts from the best point found so far under a higher bar, so the search does not end where the previous campaign's stopping rule ended it. Kernel optimization needs that restart because once an architecture is chosen, every lever inside it can price null while a better topology exists. The DSA sparse-attention generation is the worked case, where the gain came from reopening the kernel's first M_0 decision as a persistent loop rather than from any tuned parameter (Appendix I).

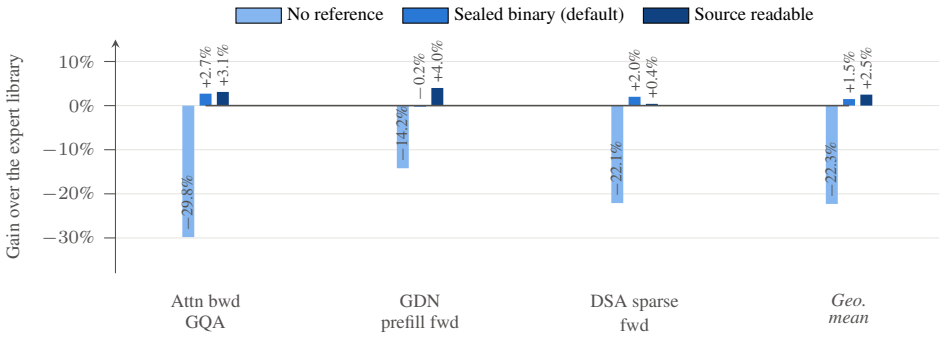


Figure 5: Reference ablation: one group per operator, one bar per arm — no reference, the sealed binary the arch agent can profile (the default), or that binary with its source readable. Every bar is the gain over the same expert library; the first two arms are capped at three meta steps.

Table 2: Reference strength on DSA's fp8 indexer forward: one campaign against each of the operator's two real references. ForgeKernel vs ref is the paired same-job gain; ForgeKernel (ms) is the median of three confirmation jobs.

	Reference	Ref. (ms)	ForgeKernel (ms)	ForgeKernel vs ref
Strong	DeepGEMM 2.6.1	1.861	1.808	+2.9%
Weak	cuDNN-frontend 1.28	2.408	2.138	+12.7%

Knowledge-base usage. Promotion compounds only because what one campaign learns survives into the next. The harness did not change between campaigns (§4.2), so the knowledge base is the one input that grew, and it is where the difference between an early campaign and a late one has to come from. Its ledgers hold 71 entries in 9 categories born across 79 runs, with 553 (entry, run) pairs in which a prescribed entry landed in the delivered kernel, and the flow grows over time: split into three consecutive periods, the earliest runs landed 4.9 foreign entries per run, the middle ones 4.1, and the latest 9.4. Invention concentrates where the gap was — the attention backward (17 entries) and DSA sparse forward (14) campaigns contributed almost half of the base — while the two largest margins of Figure 3, conv3d forward and DSA score recomputation, came from campaigns that invented nothing and landed only foreign entries. Figure 8 (Appendix I) traces the two directions campaign by campaign.

4.4 Ablation Study

Reference ablation. The reference is the one task-specific input ForgeKernel is given, so Figure 5 ablates it on 3 operators at three levels: no reference (a PyTorch implementation plus the reference's device time as a single number), the sealed binary the arch agent can run, profile and investigate, and that binary with its source readable. Without a reference ForgeKernel stalls on all three. None of them has much public design to recall — a young recurrent operator, a backward pass and a sparse kernel — so it is the reference that carries the information, and given one, under the same capped protocol, all three recover. Making its source readable helps modestly (+2.5% against +1.5% in geometric mean; the source-readable attention-backward campaign hit the meta-step cap after 11 iterations), but the sealed binary already reaches within 1% of the library on all three operators. What the arch agent mainly needs from a reference is its behavior.

Reference *strength*, by contrast, propagates. Table 2 runs the same pipeline against each of the two genuine references for DSA's fp8 indexer forward — DeepSeek's production kernel in DeepGEMM and NVIDIA's in cuDNN-frontend, 29% slower — and the resulting kernels track their reference, 18% apart for the same operator on the same GPU. Both campaigns pass their reference, the weaker one by more, yet the kernel built against cuDNN-frontend remains slower than DeepGEMM itself: the headroom above a weak reference is not enough to find what a strong one embodies. ForgeKernel rises to its reference, which is why promoting the strongest known kernel (§4.3) is how it keeps climbing.

5 Related Work

Expert kernels and kernel infrastructure. The references ForgeKernel absorbs come from expert kernel engineering — the FlashAttention line (Dao et al., 2022; Dao, 2023; Shah et al., 2024; Zadouri et al., 2026), serving libraries (Ye et al., 2025; DeepSeek-AI, 2025c;a), vendor libraries (Chetlur et al., 2014; NVIDIA Corporation, 2026a) and the infrastructure beneath them (Thakkar et al., 2026; Tillet et al., 2019; Spector et al., 2024; Wang et al., 2025b); superoptimizers and megakernel compilers (Wu et al., 2025; Cheng et al., 2025) explore parts of this space without LLMs. ForgeKernel consumes this ecosystem rather than competing with it.

Single-episode generation. Most generation systems (Yu et al., 2026) keep optimization knowledge inside one episode: test-time agent loops against compiler and profiler feedback (Chen et al., 2025; Wang et al., 2025a; Dong et al., 2025; Wei et al., 2025; Han et al., 2026; Guo et al., 2026), evolutionary search over kernel populations (Lange et al., 2025; Novikov et al., 2025; Wiedemann et al., 2026; Yan et al., 2026; Du et al., 2026b), and RL or fine-tuning that writes it into weights (Baronio et al., 2025; Li et al., 2026; 2025; Tehrani et al., 2026; Du et al., 2026a; Fu et al., 2026; AI at Meta, 2025; Yuksekgonul et al., 2026). That knowledge disappears when the episode ends or freezes when training stops. ForgeKernel keeps it in an explicit, measurement-certified artifact instead, and complements the model axis: a stronger coding agent improves every campaign.

Knowledge accumulation and reference-based learning. Recent work shares parts of this premise (Sun et al., 2026; Spoczynski et al., 2026; Ma et al., 2026; Luo et al., 2026; Chen et al., 2026; Cao et al., 2026; Zhang et al., 2026; Ye et al., 2026; Liu et al., 2026). AVO (Chen et al., 2026) leaves its knowledge implicit in the population lineage, CAKE (Ye et al., 2026) accumulates compiler rules inside a hand-built schedule IR, and Kernel Design Agents (Liu et al., 2026) released the strongest published agent-written kernels on several of our tasks, which we treat as references to surpass. No prior system combines absorbing a reference through hardware-observable behavior alone, accumulating measurement-certified knowledge across operators, and promoting accepted kernels into the next campaign's reference.

6 Discussion and Limitations

ForgeKernel gives coding agents an external architectural layer: it infers architectural decisions from hardware measurements of a sealed reference, guides a coding agent through milestones with measurement-anchored targets, and turns accepted kernels and certified findings into the references and recipes of later campaigns. Across 27 operators it reaches a +10% geometric-mean gain over expert libraries and other systems' kernels, and self-evolution adds up to a further +9.9%.

Three limitations bound that result. *A reference is required:* given only a PyTorch implementation and a device-time number, ForgeKernel stalls on all three operators of §4.4; it surpasses an implementation that exists rather than writing the first one. *The plateau is an estimate, not a proof:* self-evolution stops at the same-job noise floor, so a plateau shows that this loop found nothing more, not that nothing more exists. *Coverage is four operator families* on two NVIDIA generations and one Ascend part; sparsity patterns and memory hierarchies unlike these are untested.

The trajectory the introduction describes applies to ForgeKernel too: the parts of its harness that encode what a model cannot yet do should become unnecessary as models improve, and what should outlast them is a measurement-certified account of how a particular GPU behaves.

Reproducibility Statement

Every claim in this paper is a device-time ratio produced by the harness the paper describes, and the measurement procedure behind it is stated in §4.1 and Appendix G: reference and candidate timed in the same GPU job, a final claim taken as the median of up to 3 same-job readings, and an operator-specific correctness judge — against an fp32 reference independent of the expert kernel, with a fence rejecting any candidate that calls or forwards to that kernel — run before any timing. Every reference is pinned to the job image and re-measured in our own jobs. The per-numeric-class judge tolerances are in Appendix D, the pinned reference versions and entry points in Appendix E, the

problem shape of every task and campaign in Appendix F, and the measured cost of every campaign — arch-agent re-entries, GPU hours, active hours and wall-clock hours — in Appendix H. The `/goal` agent baseline of §2 is a stock Claude Code CLI in its default mode under the protocol stated there; no part of it is our own code. Two classes of result are not reproducible from this paper alone. The Ascend numbers are computed by CANN-Bench's hosted scoring service and can only be reproduced there. And the campaigns themselves run frontier models whose weights we do not control, so a rerun samples a different trajectory; what a rerun should reproduce is the measurement, not the trajectory, which is why every number here is a same-job ratio against a pinned reference rather than an absolute time.

AI USE STATEMENT

This work is a study of AI-generated code, so generative AI is both the subject and a tool of the research. Every GPU kernel reported as ForgeKernel's output was written by the coding agent described in §3, and every milestone curriculum by the arch agent; no author hand-wrote or hand-tuned a kernel that appears in any result. The `/goal` agent kernels of §2 were likewise produced by an unmodified commercial coding agent. All of these artifacts were admitted only after the automated correctness judges and timing protocol of §4.1 accepted them, and the authors audited every kernel counted as a win for accuracy-for-speed trades.

Generative AI was also used to assist with the harness implementation and with drafting and editing this manuscript. The authors verified the experimental code, re-ran the measurements behind every reported number, and reviewed all text. We take responsibility for the final content of this work, including its text, claims and artifacts.

REFERENCES

AI at Meta. KernelLLM. `https://huggingface.co/facebook/KernelLLM`, 2025. Model card. Accessed 2026-09.

Carlo Baronio, Pietro Marsella, Ben Pan, Simon Guo, and Silas Alberti. Kevin: Multi-turn RL for generating CUDA kernels, 2025.

Shiyi Cao, Ziming Mao, Joseph E. Gonzalez, and Ion Stoica. K-Search: LLM kernel generation via co-evolving intrinsic world model, 2026.

Terry Chen, Bing Xu, and Kirthi Devleker. Automating GPU kernel generation with DeepSeek-R1 and inference time scaling. NVIDIA Developer Blog, 2025. `https://developer.nvidia.com/blog/automating-gpu-kernel-generation-with-deepseek-r1-and-inference-time-scaling/`.

Terry Chen, Zhifan Ye, Bing Xu, et al. AVO: Agentic variation operators for autonomous evolutionary search, 2026. NVIDIA.

Xinhao Cheng, Zhihao Zhang, Yu Zhou, et al. MPK: A compiler and runtime for mega-kernelizing tensor programs, 2025.

Sharan Chetlur, Cliff Woolley, Philippe Vandermersch, Jonathan Cohen, John Tran, Bryan Catanzaro, and Evan Shelhamer. cuDNN: Efficient primitives for deep learning, 2014.

Tri Dao. Flashattention-2: Faster attention with better parallelism and work partitioning, 2023.

Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. Flashattention: Fast and memory-efficient exact attention with IO-awareness. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.

DeepSeek-AI. DeepSeek-V2: A strong, economical, and efficient mixture-of-experts language model, 2024.

DeepSeek-AI. Deepgemm: Clean and efficient FP8/FP4 GEMM kernels with fine-grained scaling. `https://github.com/deepseek-ai/DeepGEMM`, 2025a. Accessed 2026-09.

DeepSeek-AI. DeepSeek-V3.2-Exp: Boosting long-context efficiency with DeepSeek sparse attention. https://github.com/deepseek-ai/DeepSeek-V3.2-Exp/blob/main/DeepSeek_V3_2.pdf, 2025b. Tech report, September 2025.

DeepSeek-AI. Flashmla: Efficient attention kernels for DeepSeek models. https://github.com/deepseek-ai/FlashMLA, 2025c. SM90/SM100 dense and sparse attention kernels; the DeepSeek-V4.1 sparse MLA prefill and fused norm+RoPE+attention+cast kernels are upstream ba89a34. Accessed 2026-09.

Juncheng Dong, Yang Yang, Tao Liu, et al. STARK: Strategic team of agents for refining kernels, 2025.

He Du, Qiming Ge, Jiakai Hu, et al. Kernel-smith: A unified recipe for evolutionary kernel optimization, 2026a.

Weihua Du, Jingming Zhuo, Yixin Dong, et al. Adaexplore: Failure-driven adaptation and diversity-preserving search for efficient kernel generation, 2026b.

Dayuan Fu, Mohan Jiang, Tongyu Wang, et al. davinci-kernel: Co-evolving skill selection, summarization, and utilization via RL for GPU kernel optimization, 2026.

Jihu Guo, Sitian Lu, Tenghui Ma, et al. Specgen: Accelerating agentic kernel optimization with speculative generation, 2026.

Yaochen Han, Ke Fan, Hongxu Jiang, et al. EGG: An expert-guided agent framework for kernel generation, 2026.

Kimi Team. Kimi linear: An expressive, efficient attention architecture, 2025. Introduces Kimi Delta Attention (KDA).

Robert Tjarko Lange, Aaditya Prasad, Qi Sun, Maxence Faldor, Yujin Tang, and David Ha. The AI CUDA Engineer: Agentic CUDA kernel discovery, optimization and composition. Sakana AI technical report, 2025. https://pub.sakana.ai/ai-cuda-engineer/paper/.

Shangzhan Li, Zefan Wang, Ye He, et al. Autotriton: Automatic Triton programming with reinforcement learning in LLMs, 2025.

Xiaoya Li, Albert Wang, Guoyin Wang, Jiwei Li, and Chris Shum. CUDA-L1: Improving CUDA optimization via contrastive reinforcement learning. In *International Conference on Learning Representations (ICLR)*, 2026. arXiv:2507.14111.

Sihao Liu, Ligeng Zhu, et al. Kernel design agents (KDA): An agent-centric workflow for high-performance CUDA kernels. https://github.com/NVlabs/kda, 2026. NVIDIA and MIT HAN Lab; software release.

Mao Luo, Hongbin Li, Feng Lin, Hanling Yi, and Zhe Huang. Agentic kernel optimization: Generating state-of-the-art GPU kernels without hand-written CUDA, 2026.

Xing Ma, Yangjie Zhou, Wu Sun, et al. Cubridge: An LLM-based framework for understanding and reconstructing high-performance attention kernels, 2026. ACL 2026.

Alexander Novikov, Ngân Vũ, Marvin Eisenberger, et al. Alphaevolve: A coding agent for scientific and algorithmic discovery, 2025.

NVIDIA Corporation. NVIDIA H100 Tensor Core GPU architecture whitepaper. https://resources.nvidia.com/en-us-hopper-architecture/nvidia-h100-tensor-c, 2022. Accessed 2026-09.

NVIDIA Corporation. NVIDIA Blackwell architecture technical overview. https://resources.nvidia.com/en-us-blackwell-architecture, 2024. Accessed 2026-09.

NVIDIA Corporation. cuDNN frontend API. https://github.com/NVIDIA/cudnn-frontend, 2026a. Version 1.29. Accessed 2026-09.

NVIDIA Corporation. Parallel thread execution ISA, version 9.4. https://docs.nvidia.com/cuda/parallel-thread-execution/, 2026b. wgmma.mma_async, cp.async.bulk.tensor (TMA), and tcgen05 sections. Accessed 2026-09.

Jay Shah, Ganesh Bikshandi, Ying Zhang, Vijay Thakkar, Pradeep Ramani, and Tri Dao. Flashattention-3: Fast and accurate attention with asynchrony and low-precision, 2024.

Benjamin F. Spector, Simran Arora, Aaryan Singhal, Daniel Y. Fu, and Christopher Ré. Thunderkittens: Simple, fast, and adorable AI kernels, 2024.

Marcin Spoczynski, Daniel Fleischer, Moshe Berchansky, et al. Xe-forge: Multi-stage LLM-powered kernel optimization for Intel GPU, 2026.

Qitong Sun, Jun Han, Tianlin Li, et al. Kernelskill: A multi-agent framework for GPU kernel optimization, 2026.

Ali Tehrani, Yahya Emara, Essam Wissam, et al. Fine-tuning GPT-5 for GPU kernel generation, 2026. Makora.

Vijay Thakkar, Pradeep Ramani, Cris Cecka, Aniket Shivam, Honghao Lu, Ethan Yan, Jack Kosaian, Mark Hoemmen, Haicheng Wu, Andrew Kerr, et al. CUTLASS: CUDA templates for linear algebra subroutines. https://github.com/NVIDIA/cutlass, 2026. Version 4.8; includes CuTe layout algebra and the Python CuTe-DSL. Accessed 2026-09.

Philippe Tillet, Hsiang-Tsung Kung, and David Cox. Triton: An intermediate language and compiler for tiled neural network computations. In *Proceedings of the 3rd ACM SIGPLAN International Workshop on Machine Learning and Programming Languages (MAPL)*, 2019.

vincent317. Crossing the expert line without expert machinery. https://github.com/vincent317/Crossing-The-Expert-Line-Without-Expert-Machinery, 2026. GitHub repository. Accessed 2026-09.

Jianghui Wang, Vinay Joshi, Saptarshi Majumder, et al. GEAK: Introducing Triton kernel AI agent & evaluation benchmarks, 2025a.

Lei Wang, Yu Cheng, Yining Shi, Zhengju Tang, Zhiwen Mo, Wenhao Xie, Lingxiao Ma, Yuqing Xia, Jilong Xue, Fan Yang, and Zhi Yang. Tilelang: A composable tiled programming model for AI systems, 2025b.

Anjiang Wei, Tianran Sun, Yogesh Seenichamy, et al. Astra: A multi-agent system for GPU kernel performance optimization, 2025.

Nina Wiedemann, Quentin Leboutet, Michael Paulitsch, Diana Wofk, and Benjamin Ummenhofer. Kernelfoundry: Hardware-aware evolutionary GPU kernel optimization. In *International Conference on Machine Learning (ICML)*, 2026. arXiv:2603.12440.

Mengdi Wu, Xinhao Cheng, Shengyu Liu, et al. Mirage: A multi-level superoptimizer for tensor programs. In *19th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2025. arXiv:2405.05751.

Minghao Yan, Bo Peng, Benjamin Coleman, et al. Pacevolve: Enabling progress-aware consistent evolution, 2026.

Songlin Yang, Jan Kautz, and Ali Hatamizadeh. Gated delta networks: Improving Mamba2 with delta rule, 2024. ICLR 2025.

Zihao Ye, Lequn Chen, Ruihang Lai, Wuwei Lin, Yineng Zhang, Stephanie Wang, Tianqi Chen, Baris Kasikci, Vinod Grover, Arvind Krishnamurthy, and Luis Ceze. Flashinfer: Efficient and customizable attention engine for LLM inference serving. In *Proceedings of Machine Learning and Systems (MLSys)*, 2025. Best Paper Award. arXiv:2501.01005.

Zihao Ye, Yingyi Huang, Hongyi Jin, Bohan Hou, Junru Shao, Zhongming Yu, Jinqi Chen, Meghan Cowan, Shiyi Cao, Shanli Xing, Hanfeng Chen, Vinod Grover, Tianqi Chen, and Luis Ceze. CAKE: Compiler-agent co-design for frontier kernel evolution, 2026.

Yang Yu, Peiyu Zang, Chi Hsu Tsai, et al. Towards automated kernel generation in the era of LLMs, 2026. Survey.

Mert Yuksekgonul, Daniel Koceja, Xinhao Li, et al. Learning to discover at test time, 2026. TTT-Discover.

Ted Zadouri, Markus Hoehnerbach, Jay Shah, Timmy Liu, Vijay Thakkar, and Tri Dao. Flashattention-4: Algorithm and kernel pipelining co-design for asymmetric hardware scaling, 2026.

Shuoming Zhang, Qiuchu Yu, Yangyu Zhang, et al. Learning when to optimize: Verified optimization skills from expert GPU-kernel lineages, 2026. KLineage; skills extracted from open reference source, static one-shot.

Dongyun Zou, Yixin Dong, Junxian Guo, et al. KDA-v0.5 kernels for the MLSys 2026 FlashInfer AI kernel generation contest. `https://github.com/mit-han-lab/mlsys2026-flashinfer-contest-solution`, 2026. Speedups over the contest winners re-measured by the authors under the flashinfer-bench CUPTI protocol, not by the contest evaluator. Accessed 2026-09.

A Hardware Background

The two layers in full. *Architectural decisions* determine global structure through (i) the tiling hierarchy; (ii) the pipeline stage count and buffer placement; (iii) warp roles; (iv) tensor layouts in shared memory and registers; and (v) the recompute-versus-store boundary. *Tactical knowledge* determines the execution of a fixed structure through (i) warp-group MMA issuance; (ii) TMA descriptor construction; (iii) mbarrier coordination; and (iv) per-role register budgeting.

Architectural signatures in hardware behavior. (i) Occupancy and per-warp register counts reveal role specialization; (ii) TMA and L2 byte flows reveal operand staging; (iii) barrier-stall distributions reveal pipeline depth; and (iv) launch structure, block dimensions, and grid dimensions reveal kernel decomposition and the recompute-versus-store boundary.

B Gate Obligations and Milestone Validation

Cross-role hand-offs, gather-supply discipline and consumer-warpgroup phasing receive closed-vocabulary labels; each label requires stall-anchored verification and recorded ordering evidence. Each unresolved gate line receives one of three machine-checked verdicts: (i) DEFERRED, which requires evidence that the reference achieves the metric, that current levers do not change it, and a named downstream lever that receives the unresolved test; (ii) INFEASIBLE, which requires a metric-by-metric argument and evidence that forecloses the target; or (iii) N/A, which applies only to structural inapplicability. A validator checks sections, five-tuples, acyclic derivations, provenance, mechanism lines and label obligations for every milestone version. Patches are evaluated over subsequent iterations and reverted if no improvement can be attributed to them; a predefined contract assigns the asymmetry costs used when a gain on one metric trades against a loss on another.

C Milestone Construction Details

Architecture reference and library baseline. Each campaign distinguishes the *architecture reference* from the *library baseline*. The measured architecture reference sets the stop threshold and anchors every mechanism line, and the harness rejects a campaign without one; the library baseline is the best off-the-shelf call and supplies a comparison point without determining termination. The distinction prevents declaring success against whichever baseline happens to be convenient, an anchoring error an early FlashAttention backward loop made.

Decision graph. Derivation edges between milestone decisions form a DAG, which allows partial re-execution after a patch. Atomic-group annotations identify decision sets that must land in one commit because every strict subset regresses.

Evidence rules. Every milestone number carries a provenance marker, `[MEASURED, job N]` or `[ESTIMATE]`, together with its derivation, and every architectural decision names the machine-readable channel that verifies its execution; the milestone validator checks both (Appendix B). The driver that runs both loops of Figure 2 rewrites each agent's permission policy before every turn. Gate anchors consist only of reference measurements, hardware specifications, or expressions derived from those sources. Workspace isolation, milestone validation and knowledge-base lint enforce the no-code rule independently of prompt text, and ForgeKernel treats self-graded probes as a distinct bug class. The M_0 gate checks correctness, architecture predicates and mechanism lines without an absolute performance threshold; relative ratios against the reference supply the cross-job comparison at that stage. The milestone validator and knowledge-base lint run at campaign start, certification and publication.

D Correctness Judges

The per-tensor tolerance each judge applies follows the operator's numeric class: DeepGEMM's aggregate criterion for GEMMs, a relative max-abs bound for attention gradients, and a scaled tolerance plus a cosine bound for the GDN and KDA (Kimi Delta Attention (Kimi Team, 2025)) state kernels, whose small outputs would pass a fixed absolute tolerance trivially. The GDN judges further add correlated-key and long-memory inputs that catch state-decay shortcuts.

E Reference Versions and Entry Points

Library references are pinned in the job image at FlashAttention-4 (CuTe DSL), cuDNN 9 with cuDNN-frontend 1.28, DeepGEMM, FlashMLA, FlashInfer and FlashQLA, and are measured at these entry points: FlashAttention-4's forward and `_flash_attn_bwd`, FlashInfer's fused `chunk_gated_delta_rule`, FlashQLA's TileLang backward, DeepGEMM's `fp8_fp4_mqa_logits`, FlashMLA's `flash_mla_sparse_fwd`, and cuDNN-frontend's CuTe DSL radix top-k, `sparse_attention_backward`, `indexer_backward` (three kernels summed), and the `sparse_attn` + `sparse_indexer` score-recompute composition. On Ascend, each operator's reference is the fastest implementation in Huawei's current operator libraries; CANN-Bench 1.1.2 supplies the scoring baselines, and its public repository releases through v1.0.0 as of this writing.

The system references are K-Search's (Cao et al., 2026) winning kernels on its own FlashInfer-Bench workloads and the GPUMode TriMul task, the KDA-v0.5 kernels Kernel Design Agents (Liu et al., 2026; Zou et al., 2026) released for the MLSys 2026 FlashInfer contest definitions (GDN prefill and DSA sparse attention), and CAKE's (Ye et al., 2026) Kimi Delta Attention prefill kernel.

F Campaign Shapes

Table 3 lists the problem shape of every task and campaign of the main paper.

G Measurement Protocol

Timing uses CUDA events over a batch of back-to-back launches, under CUDA-graph replay or CUPTI kernel span where host dispatch would otherwise dominate. The per-tensor tolerance of each correctness judge is set for the operator's numeric class so that kernels with small outputs cannot pass trivially (Appendix D); isolation is enforced as in §3.1. The coding agent targets `sm_90a` on Hopper and `sm_100` on Blackwell.

On Blackwell, GDN and DSA are compared against Kernel Design Agents rather than a library: its GDN kernel is faster than FlashInfer's, and no library implements DSA. The Ascend campaigns are timed by CANN-Bench 1.1.2's hosted harness rather than ours — kernel-only device time from msprof's `kernel_details`, scored against the baseline and hardware bound the benchmark itself defines for each case on 910C, with compilation and accuracy folded into the score — which it computes on its own hosted service, so those numbers are reproducible only there.

Level 4 is the benchmark's hardest tier, its fused attention and RNN kernels. Against the best public submission, the lead in mean benchmark score (3.87 points) is smaller than the lead in speedup

Table 3: The problem shape of every `/goal` agent task and every ForgeKernel campaign reported in the main paper.

Operator	GPU	Shape
`/goal` agent ceiling		
attn_fwd_gqa	H100	B=2 S=4096 H_Q=16/H_KV=1 D=128 bf16 causal
fa_bwd_gqa	H100	B=10 N=4096 H_Q=16/H_KV=2 D=64 fp16 causal
gdn_prefill_fwd	H100	N=8192 h_qk=16/h_v=32 d=128 bf16
gdn_prefill_bwd	H100	N=8192 h_qk=16/h_v=32 d=128 bf16
dsa_indexer_fwd	H100	s_q=4096 s_kv=32768 H=64 D=128 fp8
dsa_indexer_topk	H100	logits fp32 [4096 x 32768] k=1024
dsa_sparse_fwd	H100	s_q=4096 s_kv=32768 h=128 d=512 topk=1024
dsa_sparse_bwd	H100	s_q=32768 s_kv=8192 h=128 d=512 topk=1024
dsa_score_recompute	H100	s_q=32768 s_kv=8192 topk=1024
dsa_indexer_bwd	H100	s_q=32768 s_kv=8192 h=64 d=128 topk=1024
ForgeKernel campaigns		
attn_fwd (GQA 16:1)	H100	B=2 S=4096 H=16/1 D=128 bf16 causal
attn_bwd (MHA)	H100	B=10 N=4096 H=16 D=64 fp16 causal
attn_bwd (GQA 8:1)	H100	B=10 N=4096 H=16/2 D=64 fp16 causal
fp8_gemm	H100	M=N=K=4096 blk 128
fp8_grouped_gemm	H100	G=8 N=K=4096 blk 128
gemm (fp16)	H100	M=N=K=8192 fp16
conv2d_fwd	H100	N=128 C=K=256 28x28 3x3 s1
conv2d_wgrad	H100	N=128 C=K=256 28x28 3x3 s1
conv2d_dgrad	H100	N=128 C=K=256 56x56 3x3 s2
conv3d_fwd	H100	N=8 C=K=256 8x28x28 3x3x3
dsa_indexer_fwd	H100	s_q=4096 s_kv=32768 H=64 D=128 fp8
dsa_indexer_topk	H100	4096x32768 fp32 k=1024
dsa_sparse_fwd	H100	s_q=4096 s_kv=32768 h=128 d=512 topk=1024
dsa_sparse_bwd	H100	s_q=32768 s_kv=8192 h=128 d=512 topk=1024
dsa_score_recompute	H100	s_q=32768 s_kv=8192 topk=1024
dsa_indexer_bwd	H100	s_q=32768 s_kv=8192 H=64 D=128 topk=1024
gdn_prefill_fwd	H100	N=8192 h=16/32 d=128 bf16
gdn_prefill_bwd	H100	N=8192 h=16/32 d=128 bf16
gqa_paged_decode	H100	workload 7a7fc288 (61535 pages)
mla_paged_decode	H100	workload 5bef8d88 (B=64; 75145 tokens)
mla_paged_prefill	H100	workload fe63f292 (B=18; 15883 q)
trimul	H100	S=1024 D=384 H=128 (case 6)
attn_bwd (MHA)	B200	B=8 N=4096 H=16 D=128 bf16 causal
fp4_grouped_gemm	B200	G=8 M=4096/group N=K=4096
sparse_mla_prefill	B200	s_q=4096 s_kv=32768 H_Q=128 H_KV=1 D=512 topk=1024 bf16 attn_sink
fused_attn_rope_cast	B200	s_q=4096 s_kv=32768 H_Q=128 H_KV=1 D=512 rope=64 topk=1024 n_wv_group=16 bf16-¿fp8 attn_sink q_norm
dsa_sparse_attn_paged	B200	workload 564007ac (T=8; 8462 pages)
gdn_prefill_varlen	B200	workload a9540651 (T=8192 N=20)
recurrent_kda_prefill	B200	T=8192 H=96 K=V=128 bf16

Table 4: Measured cost of every ForgeKernel campaign. *Arch agent* counts the arch agent's re-entries; *Active h* is agent hours plus GPU hours, and *Wall h* includes queueing on the shared cluster.

Operator	GPU	Arch agent	GPU h	Active h	Wall h
attn_fwd (GQA 16:1)	H100	6	0.9	9.9	10.5
attn_bwd (MHA)	H100	4	0.7	9.0	44.3
attn_bwd (GQA 8:1)	H100	—	7.3	56.0	122.0
fp8_gemm	H100	3	0.5	4.3	9.0
fp8_grouped_gemm	H100	6	2.0	14.1	22.7
conv2d_fwd	H100	19	1.7	30.3	64.1
conv2d_wgrad	H100	3	0.3	6.6	18.2
conv2d_dgrad	H100	3	0.5	11.3	13.1
conv3d_fwd	H100	2	2.0	8.7	27.0
dsa_indexer_topk	H100	12	0.5	15.2	20.2
dsa_sparse_fwd	H100	4	0.9	8.3	8.7
dsa_sparse_bwd	H100	3	2.3	13.8	15.5
dsa_score_recompute	H100	1	0.2	3.1	4.1
dsa_indexer_bwd	H100	6	3.8	18.4	34.4
gdn_prefill_fwd	H100	7	3.0	22.7	26.9
gdn_prefill_bwd	H100	5	6.3	28.9	36.3
gqa_paged_decode	H100	1	0.3	7.5	8.8
mla_paged_decode	H100	1	0.3	10.7	47.4
mla_paged_prefill	H100	1	1.0	5.3	5.9
trimul	H100	3	4.5	31.1	42.0
attn_bwd (MHA)	B200	9	10.1	39.2	58.7
fp4_grouped_gemm	B200	3	4.5	19.0	36.8
dsa_sparse_attn_paged	B200	2	3.4	12.5	16.9
gdn_prefill_varlen	B200	—	6.7	30.1	41.6
recurrent_kda_prefill	B200	10	34.0	97.8	134.5

Table 5: The `/goal` agent ceiling on Hopper, per kernel (§2; protocol there). *Gain* is how much faster the kernel is than its pinned expert reference; bold marks kernels that did not beat it. Entry points are in Appendix E and shapes in Appendix F.

Forward			*Backward*		
Operator	Reference	Gain	Operator	Reference	Gain
attn_fwd_gqa	FlashAttention-4	+22.8%	fa_bwd_gqa	FlashAttention-4	+3.4%
gdn_prefill_fwd	FlashInfer	+82.6%	**gdn_prefill_bwd**	FlashQLA	−25.1%
dsa_indexer_fwd	DeepGEMM	−6.0%	dsa_sparse_bwd	cuDNN-frontend	+22.0%
dsa_indexer_topk	cuDNN-frontend	+125.9%	dsa_score_recompute	cuDNN-frontend	+22.4%
dsa_sparse_fwd	FlashMLA	−22.2%	dsa_indexer_bwd	cuDNN-frontend	+12.9%

because half that scale is a compile-and-correctness floor and its performance half saturates as a candidate approaches the hardware bound. `gru` and `lstm` are the two cells below $1\times$ against the benchmark baseline: CANN-Bench ships no reference implementation for either, so their baseline is derived from the hardware bound. Every public submission is below it too — the leaders reach $0.324\times$ and $0.410\times$ — and these are the two operators where our margin over the public best is widest.

H Campaign Cost

I Additional Results

The main text states each of these results in prose and points here for the figure or table behind it.

Case study: DSA sparse attention forward. The campaign that produced ForgeKernel's DSA sparse-attention kernel stopped on its own target as soon as it matched FlashMLA, at a single non-persistent kernel whose SMs each hold one CTA at a time, so every CTA's Q load, first KV fills and output store run with no MMA to overlap them (Figure 7). Promoted to the reference, the kernel was rebuilt from its measured signature, and tuning within that topology stalled at $+4.5\%$ with every lever priced null. The arch agent's ablations then placed the exposed time at the kernel's boundary — dropping the output store alone gained 6.0% — and it reopened the first M_0 decision as a persistent loop that overlaps each work item's write-out with the next item's load. That change,

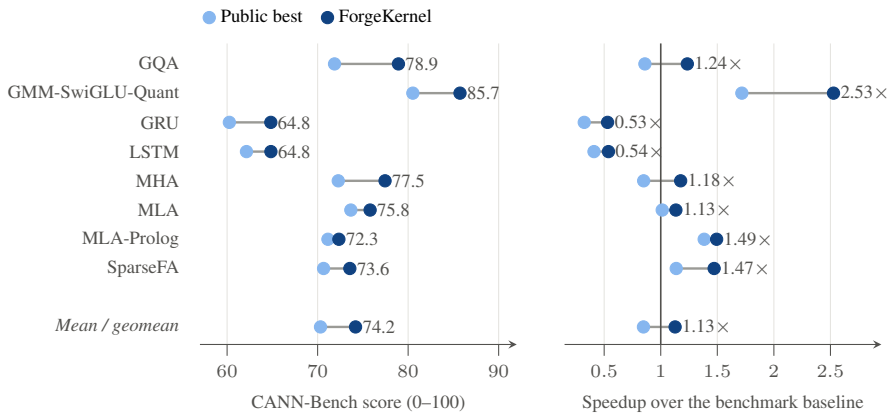


Figure 6: ForgeKernel on Ascend 910C, on all 8 level-4 operators of CANN-Bench 1.1.2. *Score* (left) is the benchmark's own metric: 0.2 compilation $+0.3$ accuracy $+0.5$ hardware-attainable performance. Speedup (right) divides both points by the same benchmark baseline, the vertical line. *Public best* is a per-operator envelope over all public submissions, read off the leaderboard on 2026-09-24, never an input to the loop.

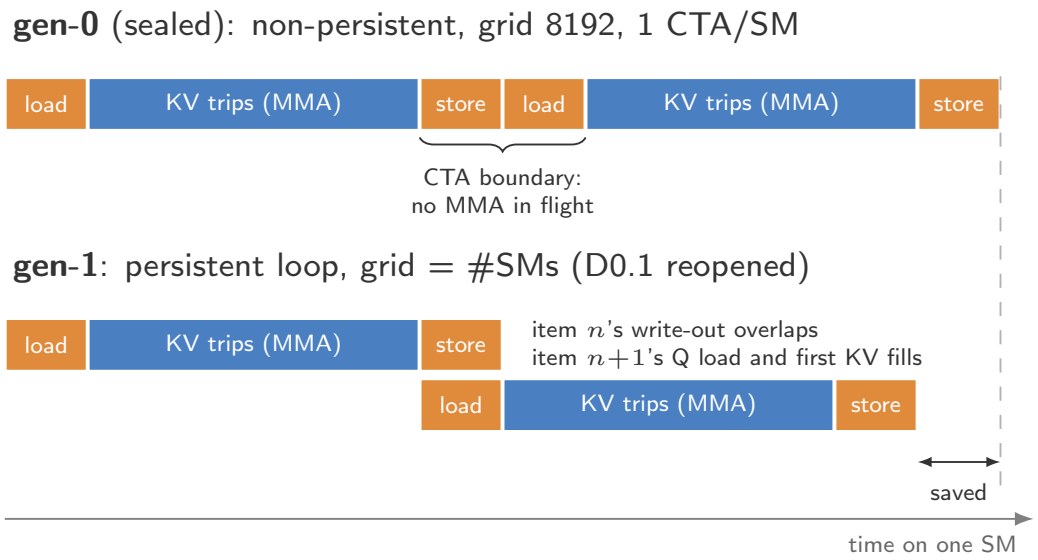


Figure 7: What self-evolution changed on DSA sparse attention forward (schematic, not to scale). Gen-0, ForgeKernel's own kernel, is non-persistent with one CTA per SM at a time; gen-1 reopens that M0 decision as a persistent loop in which item n's write-out overlaps item $n{+}1$'s load.

not a tuned parameter, carries the kernel to +7.2% over its predecessor, confirmed on a second node (§4.3).

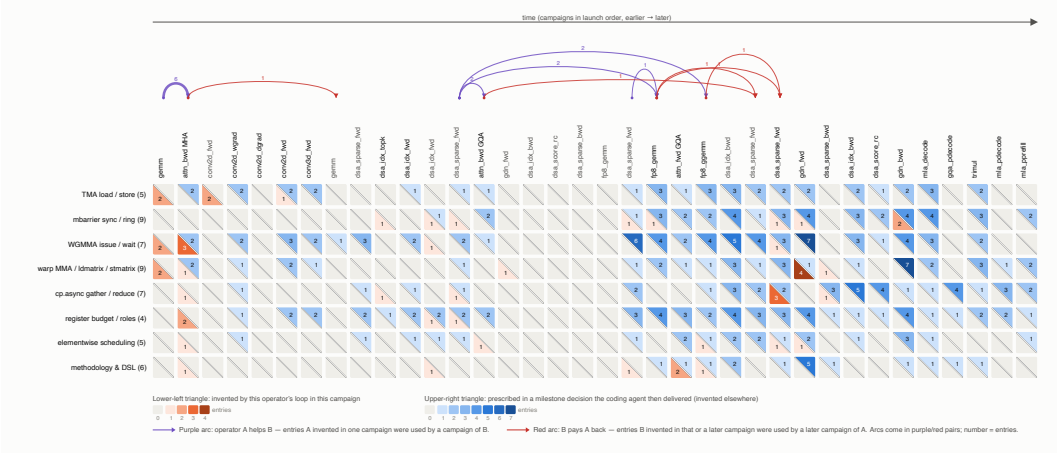


Figure 8: Knowledge provenance across 79 campaigns (one column each, in launch order) and 9 hardware-knowledge categories, behind the aggregates of §4.3. Per cell, the lower-left triangle counts entries *invented* by that campaign and the upper-right triangle entries *reused* — prescribed by its milestone and delivered by its coding agent, but invented by another operator's campaign. Purple arcs mark knowledge flowing between operators; red arcs mark a later campaign paying it back.